

Докса и логос: формализация функций неформализуемого источника в нейро-символических системах

Владимир Лапшин*

1. Введение

Формальные системы имеют фундаментальный предел. Теоремы Гёделя о неполноте показали, что любая достаточно богатая непротиворечивая система содержит утверждения, которые она не может ни доказать, ни опровергнуть [1]. Проблема остановки Тьюринга показала, что не существует общего алгоритма, который для любой программы и входа определял бы, завершится ли она [2]. Эти результаты обычно интерпретируются как абсолютные границы познания.

Мы предлагаем другую интерпретацию. Это границы *статических* систем — замкнутых множеств аксиом и правил вывода. Но человеческое мышление не является статической системой. Оно не начинается с фиксированного набора аксиом и не выводит следствия до упора. Оно постоянно расширяет, пересматривает и достраивает свои формализмы, опираясь на то, что античная традиция называла *доксой*.

Различение доксы и логоса имеет долгую историю. Уже у Ксенофана (VI в. до н. э.) *докса* ($\delta\acute{o}\xi\alpha$) обозначает знание с *неопределённой истинностной ценностью* — в отличие от уверенного знания. Парменид (V в. до н. э.) противопоставляет $\delta\acute{o}\xi\alpha$ истине ($\alpha\lambda\eta\theta\epsilon\iota\alpha$): докса — это «обманчивый порядок» чувственного мира, тогда как истина доступна только *логосу* — чистому мышлению, не доверяющему свидетельствам глаз и ушей [3]. Именно Парменид, как отмечают историки философии, впервые провёл различие между данными чувств и данными логоса и заявил, что *только второе реально и истинно*.

Платон радикализировал это различие. У него $\delta\acute{o}\xi\alpha$ — это познание *того, что*

*merfill@yandex.ru

кажется, тогда как *эпистеме* (ἐπιστήμη) — познание *того, что есть*. Докса может быть истинной или ложной, но она всегда остаётся *неустойчивой, конечной, случайной* — она «выражается отношением к предмету, а не обоснованием истины» [4]. Эпистеме же — это знание, *сопровожаемое логосом*, то есть объяснением, обоснованием, *строгим рассуждением*. Аристотель затем уточнил: докса — это *мнение*, которое может быть истинным, но не имеет *необходимого* характера; эпистеме — это знание *необходимого*, добытое через *доказательство* [5].

Таким образом, в античной традиции сложилось устойчивое различие: *докса* — это мнение, опыт, вероятностное суждение, укоренённое в чувственном и культурном; *логос* (или *эпистеме*) — это строгое, обоснованное, проверяемое знание. Докса *шире*, но *менее надёжна*. Логос *уже*, но *гарантирован*.

Мы используем эти термины в их исходном смысле, но с одним принципиальным уточнением. Докса *неформализуема*. Это не «недоформализованное знание», не «неполный логос», не «сырьё для формализации». Это *иной тип знания*. Логос — не «улучшенная докса». Это *порождение другой системы координат*. Докса — это *жизненный опыт, коллективное неформализуемое знание, культурное поле, телесность, историчность*. Она *шире* логоса, но не сводится к нему. Она не доказывает — она *предлагает*. Не *гарантирует* — а *вероятностно предполагает*. И это не недостаток. Это её *природа*. Это *основа различения*.

Попытка формализовать всё — это *избыточная формализация*, пережиток эпохи, когда математика представлялась единственным путём к строгому знанию, а всё неформализуемое считалось неполноценным. В реальности человеческое мышление всегда опирается на *оба* источника: на доксу как неисчерпаемый фон жизненного опыта и на логос как инструмент проверки и гарантии. Докса *не отменяется* логосом. Она *питает* его.

Ключевой тезис нашей работы состоит в следующем. Хотя *содержание* доксы принципиально неформализуемо, *функции* доксы — то, как она используется для расширения формальных систем — могут быть *описаны формально*. Мы вводим семейство операторов взаимодействия доксы и логоса и различаем в нём три роли: *внешние* (доксальные) операторы, которые поставляют предложение, невыводимое из теории; *внутренние* (логосные) операторы, которыми формальная система вычисляет свою реакцию на это предложение; и *протокольные* операторы, которые фиксируют, классифицируют и контролируют обмен. Только внешние операторы берут текущую формальную теорию и возвращают её *расширение*, помеченное как *предложение*, а не как *вывод*; судьбу этого предложения решает логос, который *не производит* его сам.

Этот подход не пытается «преодолеть» теоремы Гёделя. Он принимает их как

описание границ *замкнутых* систем и показывает, как *открытая* система может развиваться, используя доксу как внешний источник. Связь с понятием *оракула* — структурная: докса не является частью машины, но машина может к ней обращаться [6]. Докса *внешняя* по отношению к формальной системе; это не требует, чтобы она была невычислимой, — к этому мы вернёмся в разделах 3 и 5.

Статья устроена следующим образом. В разделе 2 мы разбираем теоретические основы: различие доксы и логоса в античной традиции и в современной логике, обзор доксистической логики, немонотонных логик, абдукции и динамических логик убеждений. В разделе 3 вводим семейство операторов взаимодействия доксы и логоса и его формальные свойства. В разделе 4 описываем конкретную реализацию этой модели. В разделе 5 обсуждаем границы подхода, его связь с гипервычислениями и практические импликации для объяснимого ИИ. Раздел 6 завершает статью.

2. Теоретические основы

В разделе 1 мы ввели различие доксы и логоса и показали его античные истоки. Здесь мы не будем повторять историю. Наша задача — показать, что это различие имеет *строгие формальные аналоги* в современной логике и что эти аналоги *релевантны* для нашей модели.

Мы не утверждаем, что эти формализмы «решают» проблему доксы. Мы утверждаем, что они *описывают отдельные аспекты* того, как докса функционирует. И эти аспекты *могут быть использованы* для построения работающей системы.

Доксистическая логика

Термин «доксистическая логика» (doxastic logic) был введён Хинтикой в 1962 году для формального моделирования *мнений и убеждений* ($\delta\acute{o}\xi\alpha$) агента, в отличие от знания ($\epsilon\pi\iota\sigma\tau\acute{\eta}\mu\eta$) [7]. Основным оператор B_α читается как «Альфа считает, что...». Доксистическая логика занимается *синхроническими* принципами — тем, как убеждения связаны между собой в данный момент.

Но она *молчит* о том, как убеждения меняются во времени. Это ограничение было осознано, и ответом на него стала *динамическая доксистическая логика* (DDL), введённая Кристером Сегербергом в 1990-х годах [8]. DDL добавляет к статической доксистической логике *модальные операторы изменения убеждений*: $[*p]q$ читается как «q выполняется после ревизии убеждений на p» [9].

Здесь важно сделать уточнение. *Докса не меняется*. Докса — это неисчерпаемый

источник жизненного опыта, культурного поля, телесного знания. Она *первична и неизменна* в том смысле, что не является объектом ревизии. Меняется *формальная система* — логос. Убеждения, о которых говорит DDL, — это *формализованный срез* доксы, зафиксированный в языке логики. DDL описывает *протокол ревизии* этого среза при поступлении новой информации. Но сама докса остаётся за *пределами* ревизии. Она — *источник*, а не *объект*.

Таким образом, DDL — это формальный аналог не доксы как таковой, а *оператора ревизии формальной системы* под воздействием доксы. Это именно то, что мы называем *функцией доксы*.

Немонотонные логики

Классическая логика *монотонна*: если $\Gamma \vdash \phi$, то $\Gamma \cup \Delta \vdash \phi$. Добавление новых аксиом *не отменяет* ранее выведенных следствий. Но формальная система, взаимодействующая с доксой, работает иначе. Новый опыт может *отменить* старые выводы. Если я знаю, что Твити — птица, я предполагаю, что он может летать. Если позже я узнаю, что Твити — пингвин, я *отменяю* вывод «Твити летает», хотя раньше он следовал из «Твити — птица».

Немонотонные логики — это формальные системы, которые моделируют именно эту *отменяемость* выводов. Классические подходы включают:

- **Логика по умолчанию** (Default Logic) *Рейтера*: правила вида «если A и нет информации об исключениях, то B », которые могут быть отменены при появлении исключения [10].
- **Циркумскрипция** (Circumscription) *Маккарти*: минимизация «ненормальных» ситуаций, позволяющая выводить «по умолчанию» [11].
- **Автоэпистемическая логика** *Мура*: моделирование убеждений агента о его собственных убеждениях [12].

Все эти формализмы признают, что *выводы формальной системы не являются окончательными*. Они *отменяемы* при поступлении новой информации из доксы. И это не «недостаток». Это *свойство* системы, которая *открыта* для взаимодействия с доксой. Немонотонные логики формализуют это свойство, но *не устраняют* его. Они показывают, как система может *работать* с отменяемыми выводами, не теряя когерентности.

Абдукция

Если немонотонные логики описывают *отмену* выводов, то *абдукция* описывает *порождение* новых гипотез. Термин был введён Чарльзом Сандерсом Пирсом (1839–1914), который различал три типа вывода: дедукцию, индукцию и *абдукцию* — «творческое порождение объяснительных гипотез» [13].

Пирс характеризовал абдукцию как рассуждение «от следствия к причине». Если мы наблюдаем факт Q , и знаем, что $P \rightarrow Q$, то абдукция позволяет *предположить* P как *возможное объяснение* Q . Это не дедукция (где P необходимо следует) и не индукция (где P вероятно следует). Это *порождение гипотезы*, которая объясняет наблюдение [14].

В современной философии науки абдукция часто отождествляется с «*выводом к наилучшему объяснению*» (Inference to the Best Explanation, IBE). Однако, как отмечают исследователи, *отождествление абдукции и IBE не является строгим*. Пирс различал *порождение гипотезы* (абдукция в узком смысле) и *выбор* наилучшей из них (IBE). Для нашей модели важно именно *порождение*: докса *предлагает* гипотезы, а формальная система *проверяет* их [15].

Абдукция *формализуема* — но только при условии, что формализуем весь «паттерн» абдуктивного вывода, включая правила порождения и обоснования гипотез. Существуют формальные системы, где абдукция добавлена к логике высказываний как *внешний оператор расширения* [22]. Это именно то, что мы называем *оператором порождения гипотез*.

Что остаётся за пределами формализации

Мы перечислили три формальных подхода: доксистическая логика (включая DDL), немонотонные логики, абдукция. Каждый из них *схватывает* один из аспектов взаимодействия доксы и формальной системы. Но *ни один из них не формализует доксу как таковую*.

Доксистическая логика формализует *структуру* убеждений как формализованного среза доксы. DDL — *протокол* ревизии этого среза. Немонотонные логики — *отменяемость* выводов при поступлении новой информации из доксы. Абдукция — *порождение* гипотез, которые докса предлагает для расширения системы. Но *содержание* доксы — её *телесность, историчность, контекстуальность* — остаётся за пределами этих формализмов.

И это не «недостаток» этих систем. Это *граница*, которую они сами признают. Немонотонные логики, например, *вычислительно сложнее* классической логики [16], и для многих из них задача установить, является ли цель следствием, *нераз-*

решима или имеет очень высокую сложность [23]. Это означает, что *не существует общего алгоритма*, который мог бы для любой теории и цели определить, является ли цель следствием. Мы можем построить *процедуру*, но не можем гарантировать, что она завершится.

Абдукция, в свою очередь, порождает *множество* гипотез, и выбор между ними — это *отдельная задача*, не имеющая формального критерия «истинности». Пропозициональная абдукция является Σ_2^P -полной задачей, то есть *сложнее*, чем NP и coNP [18], а логическая абдукция в общем случае вычислительно трудна [17].

Именно поэтому наша модель *не пытается* формализовать доксы. Она *использует* формальные системы для описания *функций* доксы, оставляя *содержание* доксы за пределами формализации. Это *не поражение*. Это *признание границы* и *построение интерфейса* к ней. Стоит подчеркнуть, что неформализуемость — это не то же самое, что невычислимость: докса не является теорией в L , но носитель, через который к ней обращаются, не обязан быть невычислимым оракулом.

Связь с нашей моделью

Мы утверждаем, что *функции* доксы — то, как она используется для расширения формальных систем — могут быть *описаны формально*. В семействе из раздела 3 каждый формальный подход, рассмотренный выше, соответствует одному оператору, и этот оператор принадлежит одной из трёх ролей:

- **Доксастическая логика** → оператор *фиксации* (что агент «держит» как убеждения). Это *протокольная* функция среды, реализуемая журналом.
- **Динамическая доксастическая логика** → оператор *ревизии формальной системы*. Это *внутренний* оператор: *логос* вычисляет, как теория меняется при поступлении новой информации из доксы.
- **Немонотонные логики** → оператор *отмены*. Это *внутренний* оператор: *логос* заново вычисляет, что следует, при появлении исключений, полученных из доксы.
- **Абдукция** → оператор *порождения гипотез*. Это *внешний* оператор: докса предоставляет гипотезу, которую формальная система проверяет.

Это *не* полный список. Мы вернёмся к нему в разделе 3, где введём *семейство операторов взаимодействия доксы и логоса*. Но уже сейчас ясно: различие доксы и логоса *не является* философской абстракцией. Оно имеет *строгие формальные аналоги* в современной логике. И эти аналоги *могут быть использованы* для построения работающей системы.

3. Семейство операторов взаимодействия доксы и логоса

В разделе 2 мы показали, что различие доксы и логоса имеет формальные аналоги в современной логике. Эти аналоги описывают отдельные аспекты взаимодействия доксы и формальной системы. Теперь мы переходим к *синтезу*. Наша задача — описать *функции*, через которые докса и логос взаимодействуют: не содержание доксы, которое неформализуемо, а *протокол* его использования. Главная мысль этого раздела в том, что этот протокол *асимметричен*: докса вносит содержание предложений, тогда как всё, что решает, фиксирует и контролирует их судьбу, принадлежит логосу и среде взаимодействия.

Основные определения и критерий роли

Пусть $F = (A, R, L)$ — формальная система с аксиомами A , правилами вывода R и языком L , и пусть $\text{Cn}(F)$ — множество её следствий. Пусть D — докса: неисчерпаемое поле жизненного опыта, культурное поле, телесное знание, историчность. D не является частью F , и она не является теорией в L .

Прежде чем определять операторы, нужны два уточнения. Во-первых, *неформализуемость* — это не невычислимость. Докса неформализуема в том смысле, что она не является системой, содержание которой можно было бы выразить аксиомами и правилами в L ; это не означает, что *носитель*, через который к ней обращаются, должен быть невычислимым оракулом. Для модели существенно лишь то, что приращение, поставляемое доксой, *внешне* по отношению к F . Во-вторых, оператор может расширять *теорию* (новые аксиомы, правила или объекты) или сам *язык*; изменение L меняет то, что считается формулой, и эти два вида расширения нельзя смешивать.

Пусть оператор — это переход $o : F \rightarrow F'$, а $\Delta = F' \setminus F$ — его приращение. Классификация операторов ведётся по *роли*, а не по положению: протокольные функции тоже лежат вне F , но они не являются источником содержания.

- **Внешний** (доксальный) оператор вносит приращение, невыводимое из F , т. е. $\Delta \not\subseteq \text{Cn}(F)$; его содержание происходит из доксы.
- **Внутренний** (логосный) оператор вычисляет F' из F и уже поставленного входа; каждый элемент Δ определяется F и этим входом.
- **Протокольный** оператор не вносит содержания и ничего не вычисляет в F ; он делает обмен корректно определённым, детерминированным и проверяемым.

Класс	Состав	Роль
Внешние (доксальные)	$o_{abd}, o_{anal}, o_{int}$	поставляют предложение, невыводимое из F
Внутренние (логосные)	$o_{rev}, o_{def}, o_{spec}, o_{ctx}$	вычисляют реакцию логоса внутри F
Протокольные (среда)	журнал/фиксация, классификация, объ- явление, заземление, волны	делают обмен корректно опре- делённым и проверяемым

Внешние операторы (доксальные)

Оператор порождения (o_{abd}). Берёт F и наблюдение Q и возвращает F' , содержащую гипотезу P , которая объясняет Q . Формально $o_{abd} : F \times Q \rightarrow F'$, где $F' = F \cup \{P\}$ и $P \notin \text{Cn}(F)$. Это формальный аналог абдукции. Гипотеза *предлагается*, а не выводится; логос проверяет её на совместимость с F и на объяснительную силу [22].

Оператор аналогии (o_{anal}). Берёт знакомую область F_1 и новую область F_2 и возвращает F'_2 , в которой структура F_1 перенесена на F_2 . Выбор структурного отображения внешний; логос лишь проверяет, сохраняется ли оно.

Оператор интуиции (o_{int}). Берёт F и цель ϕ и возвращает ответ ϕ' , который не следует из F . Это оператор, ближе всего стоящий к оракулу. Докса *даёт* ответ; логос проверяет его, если может, иначе откладывает как гипотезу. Оракулоподобен здесь *источник* ответа, а не носитель, который вполне может быть вычислимым.

Внутренние операторы (логосные)

Эти операторы часто принимают за функции доксы, потому что они запускаются её входом. На самом деле их вычисляет логос; вход — лишь их аргумент.

Оператор ревизии (o_{rev}). По F и новой информации p (поставленной извне) возвращает F' , в которой убеждения пересмотрены с учётом p . Формально $o_{rev} : F \times P \rightarrow F'$. Это формальный аналог ревизии AGM [21] и динамической докстической логики (DDL). Докса поставяет p ; логос решает, как p влияет на F .

Оператор отмены (o_{def}). По F и исключению e (поставленному извне) возвращает F' , в которой ранее выведенные следствия отменены. Формально $o_{def} : F \times E \rightarrow F'$. Это формальный аналог немонотонных логик. Отмена не означает удаления

аксиом: то, что было выводимо, перестаёт быть выводимым при новой конфигурации.

Оператор специфичности (o_{spec}). По F и двум конфликтующим правилам r_1, r_2 возвращает F' , в которой выбрано более специфичное. Формально $o_{\text{spec}} : F \times \{r_1, r_2\} \rightarrow F'$. Специфичность — это отношение, *вычисляемое логосом* по иерархии F ; докса его не поставляет. Это формальный аналог предпочтения по специфичности в немонотонных логиках.

Оператор применения контекста (o_{ctx}). По F и объявленному контексту Γ возвращает F' , в которой значения термов переопределены в соответствии с Γ . Это формальный аналог ситуационной логики. Контекст *объявляется* протоколом (см. ниже); его *применение* внутренне.

Протокольные операторы (среда)

Эти функции не принадлежат ни доксе, ни логосу. Они не вносят содержания и ничего не вычисляют в F , но без них взаимодействие нельзя провести корректно.

Журнал и фиксация (o_{fix}). Каждый шаг фиксируется в журнале: идентификатор оператора, волна, обоснование, источник и результат (принято, отвергнуто, отложено). Журнал — это протокол взаимодействия, не часть ни доксы, ни логоса. Фиксация здесь означает выбор формализованного среза доксы, к которому обращаются; это учётное действие среды, а не функция доксы.

Детерминированная классификация. Каждое предложение классифицируется по фиксированным правилам (см. роль логоса ниже), независимо от «настроения»; именно это делает обмен воспроизводимым.

Объявление контекста и фрагмента. Контекст Γ или требуемый фрагмент логики *объявляется* параметром запуска; его применение внутренне (см. o_{ctx} выше).

Проверки заземления. Проверки дословных цитат и другие структурные свидетели удостоверяют, что предложение закреплено в источнике.

Волновой протокол. Правило «не более одного предложения за акт обращения» делает каждый акт проверяемым и оставляет след того, где и почему изменилась теория.

Свойства семейства

Внешние операторы не коммутируют, и порядок их применения существенен. Пусть F содержит `is_a(tweety, bird)` и дефолт `bird  $\Rightarrow$  fly`, и пусть докса может предложить `is_a(tweety, penguin)`, после чего у логоса появляется более

специфичный дефолт $\text{penguin} \Rightarrow \neg \text{fly}$. Если применить o_{abd} первым, а o_{spec} вторым, выбирается пингвиний дефолт и fly отменяется. Если o_{spec} применить первым, конкурирующего правила нет и выводится fly ; последующий o_{abd} тогда не выбирает, а вынуждает *ревизию*. Одни и те же операторы в разном порядке дают разные теории: процесс историчен, и каждый шаг меняет пространство возможных следующих шагов.

Семейство не сводится к одному оператору. Выражение $o_1 \cup o_2$ не является оператором: поскольку эффект o_2 зависит от того, применён ли уже o_1 , не существует функции только от исходного F , которая схватывала бы эту пару.

Внешние операторы не гарантируют результата. o_{abd} может предложить противоречивую гипотезу, o_{anal} — неподходящее отображение, o_{int} — ошибочный ответ. Это не дефект, а природа доксы: она предлагает, но не гарантирует. Внутренние операторы, напротив, являются детерминированными функциями F и своего входа; протокольные операторы гарантируют детерминизм и проверяемость, а не истину.

Процесс рассуждения

Рассуждение — это *последовательность* расширений:

$$F_0 \xrightarrow{o_1} F_1 \xrightarrow{o_2} F_2 \xrightarrow{o_3} \dots$$

где каждый шаг — *акт* обращения к доксе, помеченный оператором, который его опосредует; фиксацию, классификацию и применение каждого акта выполняет протокол. Не «вычисление», а *обращение* к источнику.

Каждый шаг *фиксируется* в журнале: идентификатор оператора, волна, обоснование, источник и результат (принято / отвергнуто / отложено). Журнал — это *протокол* взаимодействия доксы и логоса. Он *не является* частью ни доксы, ни логоса. Он — *интерфейс*, протокольный оператор, а не функция доксы.

Роль логоса

Логос *не пассивен*. Он *классифицирует* каждое предложение доксы:

- **derivable** — предложение уже выводится из F . Принимается как *нарратив*.
- **cited** — предложение заземлено *цитатой* из текста. Добавляется в аксиомы.
- **hypothesis** — предложение ново. Записывается в журнал с меткой H .

- **rejected** — предложение не проходит проверку. Отбрасывается с кодом причины.

Классификация — это протокольная функция; вердикт, который она выносит, принадлежит логосу. Классификация *детерминирована*: она зависит от *структуры F* и *содержания* предложения, а не от «настроения» логоса. Это *гарантия* того, что логос не «поддаётся» доксе. Он *принимает* только то, что *может* принять. И *честно* говорит, когда *не может*.

Границы модели

Модель **не претендует** на формализацию доксы. Она *не говорит*, что такое докса «на самом деле». Она *не утверждает*, что докса «сводится» к операторам. Она *описывает* только *функции*, через которые докса *взаимодействует* с логосом.

Модель **не гарантирует** полноты. Список операторов *открыт*. Могут быть операторы, которые мы *не выделили*. Могут быть аспекты доксы, которые *не выражаются* через операторы. Модель *не утверждает* обратного.

Модель **не решает** проблему неполноты. Она *принимает* её как *границу* логоса и показывает, как *открытая* система может *развиваться*, используя доксу как *внешний источник*. Это *не преодоление* Гёделя, это *жизнь* с Гёделем, не «после» него, а *вместе* с ним.

4. Реализация: Ankyra

В разделе 3 мы ввели семейство операторов взаимодействия доксы и логоса. Здесь мы покажем, что эта модель не является теоретической конструкцией: она реализована в работающей системе, гибридном нейро-символическом движке рассуждений Ankyra [19]. Мы опишем, как реализована каждая из трёх ролей — внешние операторы, внутренние операторы и протокол, — и отметим, что полное описание архитектуры — предмет отдельной работы.

Докса — это LLM

Ключевое утверждение, которое мы хотим сделать явным: *докса в Ankyra — это большая языковая модель*. Не «аналогия». Не «метафора». Не «философская иллюстрация». LLM *и есть* носитель, через который к доксе обращаются в машинной форме.

Из предыдущего изложения у читателя могло бы сложиться мнение, что докса —

это нечто мистическое и невыразимое, доступное только человеческому сознанию. Мы утверждаем обратное. Докса — это *поле знаний*, доступное через язык. LLM обучена на текстах, т. е. на языке, а язык — это не просто средство передачи информации. Это *картина мира* — линза, задающая сетку восприятия. LLM представляет собой гипертрофированную версию этой линзы. Она обучена на текстах, которые уже содержат знания о мире, и реконструирует реальность через паттерны языка. Это снимок снимков — язык о языке, и именно поэтому LLM даёт возможность рассматривать её как доксу: она *не владеет истиной*, но *полна жизни*. LLM как докса в сжатом виде: жизненный опыт человечества, сжатый в матрицы весов; культурное поле, доступное через предсказание следующего токена; историчность, застывшая в вероятностях.

Важно не перечитывать оракульную аналогию слишком буквально. LLM — вычислимая функция своего входа, и модель нигде не требует невычислимого источника. Докса-носителем её делает не отсутствие вычислимости, а *внешность* по отношению к формальной теории: приращение, которое она предлагает, невыводимо из аксиом и правил в L . Неформализуемость доксы здесь означает именно то, что она не является теорией на языке логоса, — а не то, что она лежит за пределами вычислимости.

Принцип: LLM предлагает, движок решает

Ядро Ankyra — различение двух участников. LLM выполняет роль доксы: читает текст, разбирает его на структуру и в цикле рассуждений предлагает дополнительные шаги. Символьный движок выполняет роль логоса: превращает структуру в формальную теорию, достраивает её выводом, сопоставляет с вопросом и удостоверяет то, что действительно следует.

Граница абсолютна. LLM никогда не владеет истиной. В теорию не попадает ничего, что движок не сможет обосновать либо дословной цитатой из текста, либо явной пометкой гипотезы.

Контракт предложения

Каждое утверждение, которое LLM хочет добавить, классифицируется детерминированно в одну из четырёх категорий: *derivable* (уже следует из теории), *cited* (подкреплено дословной цитатой), *hypothesis* (знание, которого в тексте нет), *rejected* (не проходит проверку). Это прямая реализация оператора порождения гипотез (o_{abd}): LLM предлагает, движок классифицирует, ничто не принимается без основания. Различение между *cited* и *hypothesis* делает ответы честными: вывод на цитатах — *proven*, вывод на предположениях — *proven_under(H)*

с перечислением этих предположений. Ответ никогда не бывает сильнее стоящих за ним оснований.

Цикл рассуждений

Задача проходит четыре фазы: *фаза 0* — декомпозиция (LLM разбирает текст на структуру), *фаза 1* — решение (движок достраивает теорию), *фаза 2* — цикл с подсказками (LLM предлагает по одному шагу за раз, если теория не решает вопрос), *фаза 3* — объяснение (цепочка собирается механически из истории доказательства).

Цикл с подсказками (фаза 2) организован в *волны*. Волна — это один полный акт обращения к доксе: движок проверяет текущее состояние теории и формулирует пробелы, LLM получает подсказку с этими пробелами и предлагает *ровно одно* типизированное предложение, движок классифицирует это предложение и применяет или отбрасывает его, после чего волна фиксируется в журнале и цикл начинается заново. Ограничение «одно предложение за волну» принципиально: оно делает каждое обращение к доксе проверяемым и оставляет след, по которому можно восстановить, на каком именно шаге и почему изменился ответ.

Цикл с подсказками реализует процесс обращения к доксе из раздела 3: каждая волна — это один акт $F_i \xrightarrow{o} F_{i+1}$, опосредованный *внешним* оператором, тогда как фиксацию, классификацию и применение выполняет протокол.

Журнал

Каждый шаг фиксируется в журнале: идентификатор оператора, волна, обоснование, источник, результат. Журнал — это протокол взаимодействия доксы и логоса, не часть ни того, ни другого. Он реализует *протокольную* роль семейства (журнал и фиксация): фиксирует каждый акт обращения к доксе и делает процесс проверяемым и воспроизводимым. Это не функция доксы.

Монотонная база и движущийся ответ

База — аксиомы, правила, гипотезы — только растёт: посылки не удаляются, цель не ослабляется. Но ответ может меняться по мере поступления новой информации. Каждое изменение записывается как *Revision* с указанием волны и частных предположений. Это *внутренний* оператор семейства (ревизия, o_{rev}): докса поставяет информацию, логос вычисляет, как она влияет на теорию, и решение фиксируется, а не скрывается.

Движки и объявленный фрагмент

Ankura — не монолитный решатель, а оркестратор формализаций и решающих процедур. Разные виды задач требуют разных логик: хорновские клаузы (L0), стратифицированное отрицание (L1), дизъюнкция и кванторы (L2), конечнодоменные ограничения (L3), арифметика (L4), дефолты со специфичностью (D). Эти движки — *внутренние* операторы семейства (логосная сторона): L1 и D реализуют соответственно отмену (o_{def}) и специфичность (o_{spec}), а L2 реализует разбор случаев внутри решающей процедуры. Ни один из них не является источником содержания.

Выбор процедуры осуществляется через контракт объявленного фрагмента: движок исследует построенные теорию и запрос и выводит набор требуемых признаков. Если фрагмент укладывается в доступные возможности — выбирается соответствующая процедура; если нет — результатом становится именованный пробел `out_of_fragment`, а не тихое понижение до более слабой логики. Объявление фрагмента — это *протокольная* функция; его применение внутренне, и система честно сообщает о несоответствии, а не угадывает.

Честность по построению

Ценность системы держится на гарантиях, обеспечиваемых механически: в теории нет фактов, принадлежащих модели; вопрос никогда не является источником; в детерминированном коде нет разбора естественного языка; предположив P , нельзя опровергнуть $\neg P$; ответ, использовавший предположения, всегда помечен `proven_under(H)`. Это реализация свойства прозрачности интерфейса: логос принимает только то, что может принять, и честно говорит, когда не может.

Связь с концептуальной моделью

Ankura реализует три роли семейства из раздела 3. *Внешние*: контракт предложения (оператор порождения, o_{abd}). *Внутренние*: ревизия (учёт Revision), отмена (L1), специфичность (D). *Протокольные*: журнал, детерминированная классификация, проверки заземления цитат, волновой протокол и объявление фрагмента. Два внешних оператора пока не реализованы: аналогия (o_{anal}) и интуиция (o_{int}). Система не претендует на полноту: она реализует то, что может реализовать честно, и честно говорит о том, чего не может.

5. Обсуждение

Мы ввели различие доксы и логоса, показали его формальные аналоги и предложили семейство операторов взаимодействия доксы и логоса, реализованное в Ankyra. Теперь необходимо обсудить границы этого подхода: что он объясняет, чего не объясняет, и какие следствия из него вытекают.

Что модель объясняет

Модель объясняет, каким образом формальная система может развиваться, не нарушая собственной корректности. Теоремы Гёделя и Тьюринга описывают пределы замкнутых систем. Наша модель показывает, что открытая система — та, которая взаимодействует с доксой через явный интерфейс, — может расширяться без того, чтобы это расширение было выводом внутри неё самой. Расширение приходит извне. Логос его не производит. Он его принимает, отвергает или откладывает.

Это не «преодоление» теорем о неполноте. Это их принятие как описания границы и построение интерфейса к тому, что лежит за границей. Неполнота не устраняется, а *переносится*: полученная теория F' снова неполна, если она непротиворечива и достаточно богата, но точка, в которой система останавливается, сместилась с замыкания на интерфейс. Мы не утверждаем, что докса «решает» проблему остановки или неполноты. Мы утверждаем лишь, что система, которая честно признаёт границу и обращается к внешнему источнику, может продолжать работать там, где замкнутая система остановилась бы.

Границы подхода

Модель не формализует содержание доксы. Мы вводим это как принципиальное ограничение, а не как временную трудность. Докса неформализуема по природе — не потому, что «мы ещё не нашли способ», а потому, что она не является системой. Она — поток, опыт, культурное поле, телесность, историчность. Формализовать её содержание означало бы превратить её в логос, то есть уничтожить то, чем она является.

Модель также не гарантирует, что набор операторов, который мы выделили, полон. Мы рассматриваем его как открытый список. Операторы аналогии и интуиции, не реализованные в текущей версии, показывают, что пространство операторов шире того, что мы описали. Могут существовать операторы, которых мы не видим. Это не недостаток модели, а следствие её предмета.

Наконец, модель не даёт критерия «правильного» выбора между конкурирующими предложениями доксы. Если два оператора порождают несовместимые гипотезы, система фиксирует конфликт, но не решает его автоматически. Выбор остаётся за *Свидетелем* — человеком или внешней инстанцией, которая несёт ответственность за выбор и для которой система существует. Модель намеренно не автоматизирует этот шаг: никакой формальный критерий не решает спор между несовместимыми предложениями доксы.

Связь с гипервычислениями

В теории вычислений понятие оракула было введено Тьюрингом для описания машины, которая может получать ответы на вопросы, не решаемые её собственной процедурой [6]. Оракул не является частью машины, но машина может к нему обращаться. Докса в нашей модели выполняет роль, структурно близкую к оракулу: она не входит в формальную систему, но система может задавать ей вопросы и получать предложения.

Однако есть важное различие. Оракул в теории вычислений даёт *истинные* ответы на вопросы определённого класса. Докса не даёт истинных ответов. Она даёт *предложения*, которые могут быть истинными, ложными или неопределёнными. Именно поэтому интерфейс к доксе включает классификацию: предложение входит в систему не как истина, а как гипотеза, требующая проверки. Логос не доверяет доксе. Он её использует.

Это различие принципиально. Гипервычисления в строгом смысле остаются теоретическим понятием: неизвестно, реализуемы ли они физически [20]. Наша модель не претендует на выход за пределы вычислимости и не требует, чтобы источник был невычислимым. Она описывает, как система, оставаясь в пределах вычислимости, может расширяться за счёт источника, *внешнего* по отношению к ней, — вычислим ли он, как LLM, или нет.

Практические следствия

Для объяснимого искусственного интеллекта модель даёт один ясный принцип: *ответ не должен быть сильнее своих оснований*. Если система вывела заключение из цитат — это одно. Если из предположений — другое. Если из смеси — третье, и это третье должно быть явно помечено. Пользователь имеет право знать, на чём стоит ответ.

Для верификации систем это означает, что проверяемость не требует полной формализации. Достаточно, чтобы каждый шаг был прослеживаем до источника, а

каждое расширение фиксировалось в журнале. Докса остаётся неформализуемой, но её *след* в системе — формален.

Для архитектуры гибридных систем это означает, что разделение между нейросетевой и символьной частью не является инженерным компромиссом. Это структурное разделение, отражающее различие доксы и логоса. Нейросетевая часть не «слабее» символьной. Она делает то, что символьная делать не может. Символьная часть не «жёстче» нейросетевой. Она делает то, что нейросетевая не должна. Каждая сторона сильна в своём.

Что остаётся открытым

Модель оставляет открытыми несколько вопросов. Как соотносятся операторы разных ролей — существуют ли законы, связывающие внешние, внутренние и протокольные шаги, тождества, приоритеты? Как измерить эффективность оператора — по глубине изменения теории, по числу принятых предложений, по вкладу в итоговый ответ? Как отличить оператор, который действительно расширяет систему, от оператора, который лишь переформулирует уже известное? Эти вопросы не имеют ответа в рамках текущей работы. Они указывают направление дальнейшего исследования.

6. Заключение

Мы предложили интерпретацию теорем о неполноте не как абсолютных границ познания, а как границ статических систем. Человеческое мышление не является статической системой: оно развивается, опираясь на доксу — неформализуемый источник жизненного опыта, культурного поля и телесного знания. Различение доксы и логоса имеет формальные аналоги в современной логике, но ни один из них не формализует доксу как таковую.

Наш вклад состоит во введении семейства операторов взаимодействия доксы и логоса — формального интерфейса между неформализуемым источником и формальной системой, — и в том, что этот интерфейс *асимметричен*: внешние операторы поставляют содержание, внутренние решают, а протокольные делают обмен проверяемым. Содержание доксы неформализуемо, но эти функции могут быть описаны формально. Мы реализовали эту модель в системе Ankyra, где большая языковая модель выполняет роль доксы, а символьный движок — роль логоса [19]. Принцип «LLM предлагает, движок решает» обеспечивает прозрачность: ничто не попадает в теорию без обоснования, и ответ никогда не бывает сильнее стоящих за ним оснований.

Мы не утверждаем, что предложили решение проблемы неполноты. Мы утверждаем, что показали, как система может работать с этой проблемой, не пытаясь её преодолеть. Докса не отменяет логос — она его питает. И в этом взаимодействии не поражение формализации, а её расширение.

Список литературы

- [1] Gödel K. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I // Monatshefte für Mathematik und Physik. 1931. Bd. 38. S. 173–198.
- [2] Turing A. M. On Computable Numbers, with an Application to the Entscheidungsproblem // Proceedings of the London Mathematical Society. 1936. Ser. 2, Vol. 42. P. 230–265.
- [3] Diels H., Kranz W. Die Fragmente der Vorsokratiker. Berlin: Weidmann, 1903–1910.
- [4] Платон. Теэтет // Платон. Собрание сочинений: в 4 т. М.: Мысль, 1990–1994. Т. 2.
- [5] Аристотель. Никомахова этика // Аристотель. Собрание сочинений: в 4 т. М.: Мысль, 1975–1983. Т. 4.
- [6] Turing A. M. Systems of Logic Based on Ordinals // Proceedings of the London Mathematical Society. 1939. Ser. 2, Vol. 45. P. 161–228.
- [7] Hintikka J. Knowledge and Belief: An Introduction to the Logic of the Two Notions. Ithaca: Cornell University Press, 1962.
- [8] Segerberg K. The Basic Dynamic Doxastic Logic of AGM // Frontiers in Belief Revision / Ed. by M.-A. Williams, H. Rott. Dordrecht: Kluwer, 2001. P. 57–84.
- [9] Segerberg K. Some Completeness Theorems in the Dynamic Doxastic Logic of Iterated Belief Revision // The Review of Symbolic Logic. 2010. Vol. 3, No. 1. P. 1–24.
- [10] Reiter R. A Logic for Default Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 81–132.
- [11] McCarthy J. Circumscription—A Form of Non-Monotonic Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 27–39.
- [12] Moore R. C. Semantical Considerations on Nonmonotonic Logic // Artificial Intelligence. 1985. Vol. 25, No. 1. P. 75–94.

- [13] Пирс Ч. С. Начала прагматизма. СПб.: Алетейя, 2000.
- [14] Peirce C. S. Lectures on Pragmatism (1903) // The Collected Papers of Charles Sanders Peirce. Cambridge: Harvard University Press, 1931–1958. Vol. 5.
- [15] Niiniluoto I. Defending Abduction // Philosophy of Science. 1999. Vol. 66, No. 3. P. S436–S451.
- [16] Виньков М. М., Фоминых И. Б. Темпоральные немонотонные логические системы: взаимосвязи и вычислительная сложность // Искусственный интеллект и принятие решений. 2008. № 4. С. 19–25.
- [17] Eiter T., Gottlob G. The Complexity of Logic-Based Abduction // Journal of the ACM. 1995. Vol. 42, No. 1. P. 3–42.
- [18] Eiter T., Gottlob G. Propositional Abduction is Σ_2^P -Complete // Information Processing Letters. 1995. Vol. 55, No. 4. P. 213–218.
- [19] Ankyra: A domain-agnostic neuro-symbolic reasoning engine. GitHub repository. URL: <https://github.com/merfill/ankyra>
- [20] Cotogno P. Hypercomputation and the Physical Church-Turing Thesis // The British Journal for the Philosophy of Science. 2003. Vol. 54, No. 2. P. 181–203.
- [21] Alchourrón C. E., Gärdenfors P., Makinson D. On the Logic of Theory Change: Partial Meet Contraction and Revision Functions // The Journal of Symbolic Logic. 1985. Vol. 50, No. 2. P. 510–530.
- [22] Kakas A. C., Kowalski R. A., Toni F. Abductive Logic Programming // Journal of Logic and Computation. 1992. Vol. 2, No. 6. P. 719–770.
- [23] Cadoli M., Schaerf M. A Survey of Complexity Results for Non-Monotonic Logics // Journal of Logic Programming. 1993. Vol. 17, No. 2–4. P. 127–160.